

6장+7장

1. 순환신경망

: 순서가 있는 연속적인 데이터를 처리하는 데 적합

- 각 시점의 데이터가 이전 시점의 데이터와 독립적 X
- 자연어는 연속형 데이터의 특성을 가지고, t번째 단어는 t-1번째까지의 단어에 영향을 받음
- 순환신경망은 이전 시점의 은닉시점을 입력으로 받아 현재 시점의 은닉 상태를 계산 → 재귀적 특성

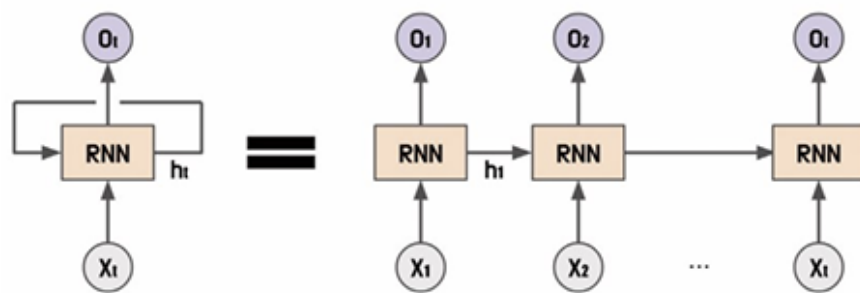


그림 6.12 순환 신경망의 셀

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터를 처리

1. 일대일구조 : 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망

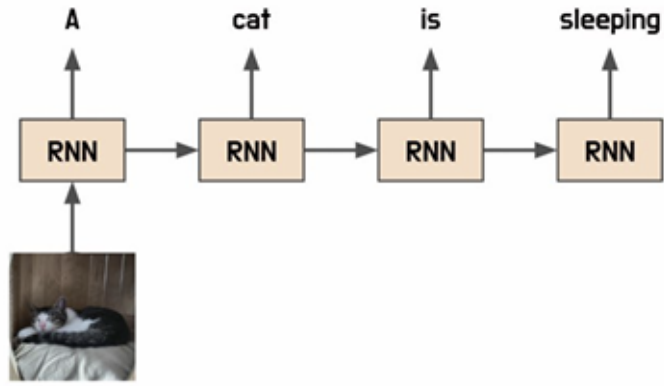


그림 6.13 순환 신경망의 일대다 구조

2. 다대일구조 : 여러 개의 입력시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망

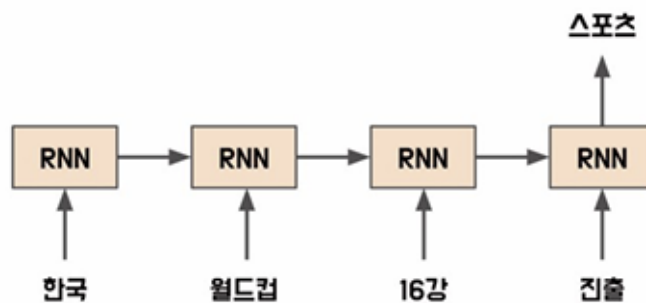


그림 6.14 순환 신경망의 다대일 구조

3. 다대다 구조 : 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 구조

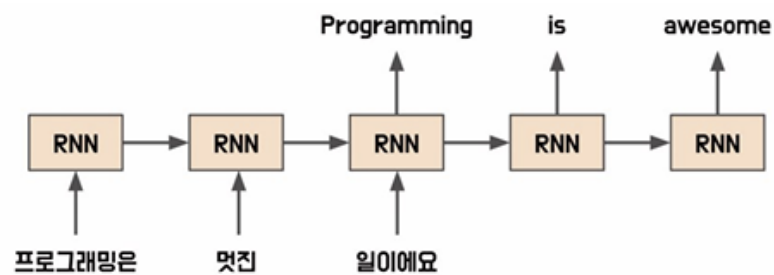


그림 6.15 순환 신경망의 다대다 구조

4. 양방향 순환 신경망(BiRNN) : 시간 방향을 양방향으로 처리할 수 있도록 고안됨

- 일반적인 RNN은 과거에서 현재 방향으로 데이터를 처리하기 때문에 현재 시점보다 뒤에 등장하는 정보는 활용X

- 하지만 자연어에서는 현재 단어의 의미를 이해하기 위해 앞 문맥뿐 아니라 뒤 문맥도 중요한 경우가 많음.
- 이를 위해 사용하는 구조가 Bidirectional RNN(BiRNN)
- 하나의 RNN은 sequence를 정방향으로 처리하고, 다른 RNN은 역방향으로 처리함.
- 두 방향에서 얻은 정보를 함께 사용하여 각 시점의 representation을 생성함.

장점

- 이전 정보와 이후 정보를 모두 고려할 수 있어 문맥을 더 풍부하게 표현할 수 있음.

한계

- 이후 시점의 데이터가 필요한 구조이므로 미래 정보가 존재하지 않는 실시간 예측에는 적합하지 않을 수 있음.
- 양방향으로 계산하기 때문에 단방향 RNN보다 계산량도 증가함.

5. 다중순환신경망 (Stacked Recurrent Neural Network) : 여러 개의 순환 신경망을 연결해 구성한 모델 → 각 순환신경망이 서로 다른 정보를 처리하도록 처리

- 하나의 RNN layer만 사용하는 것이 아니라 여러 개의 RNN layer를 쌓은 구조임.
- 아래 layer의 출력이 위 layer의 입력으로 전달됨.
- 층이 깊어질수록 sequence의 정보를 단계적으로 추상화하여 더욱 복잡한 특징을 학습할 수 있음.

순환신경망 클래스 : 파이토치

- 입력 특성 크기와 은닉 상태 크기를 입력해 순환신경망 구성 가능
- 활성화 함수 종류 설정, 과대적합 방지를 위한 드롭아웃 설정

문제점

- 장기의존성 문제
- 기울기 소실 및 기울기 폭주

⇒ 장단기 메모리 사용

2. LSTM

LSTM의 등장 배경 : 기본 RNN의 장기 의존성 문제를 개선하기 위해 제안된 구조

- 기본 RNN처럼 Hidden State만 사용하는 것이 아니라, 장기간 정보를 유지하기 위한 Cell State를 별도로 사용
- 또한 어떤 정보를 유지하고, 삭제하고, 출력할 것인지를 결정하기 위해 여러 개의 Gate를 사용
- 셀 상태(Cell State) : LSTM에서 장기간 정보를 전달하는 핵심 통로, Gate들이 Cell State에 어떤 정보를 유지하고 제거할지를 조절
- 망각 게이트(Forget gate) : 이전 Cell State에 저장되어 있는 정보 중 어떤 정보를 버릴지 결정, 필요하지 않은 정보는 제거하고, 필요한 정보는 유지하도록 조절
- 기억 게이트(Input gate) : 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거침
- 출력 게이트(output gate) : 현재시점 입력+이전상태의 은닉 + 업데이트된 메모리셀로 현재 시점의 출력값

3. Transformer

Transformer 등장 배경

- Transformer는 RNN이나 CNN 대신 Attention Mechanism을 중심으로 sequence를 처리하는 모델
- 기존 RNN 계열 모델은 입력을 순서대로 처리하기 때문에 sequence가 길어질수록 학습 속도가 느려지고 병렬 처리가 어려움
- LSTM을 통해 장기 의존성 문제를 어느 정도 완화할 수 있지만, 여전히 데이터를 시점별로 순차 처리해야 한다는 구조적 한계
- Transformer는 이러한 순환 구조를 제거하고 Attention을 통해 sequence 내 단어 사이의 관계를 직접 계산

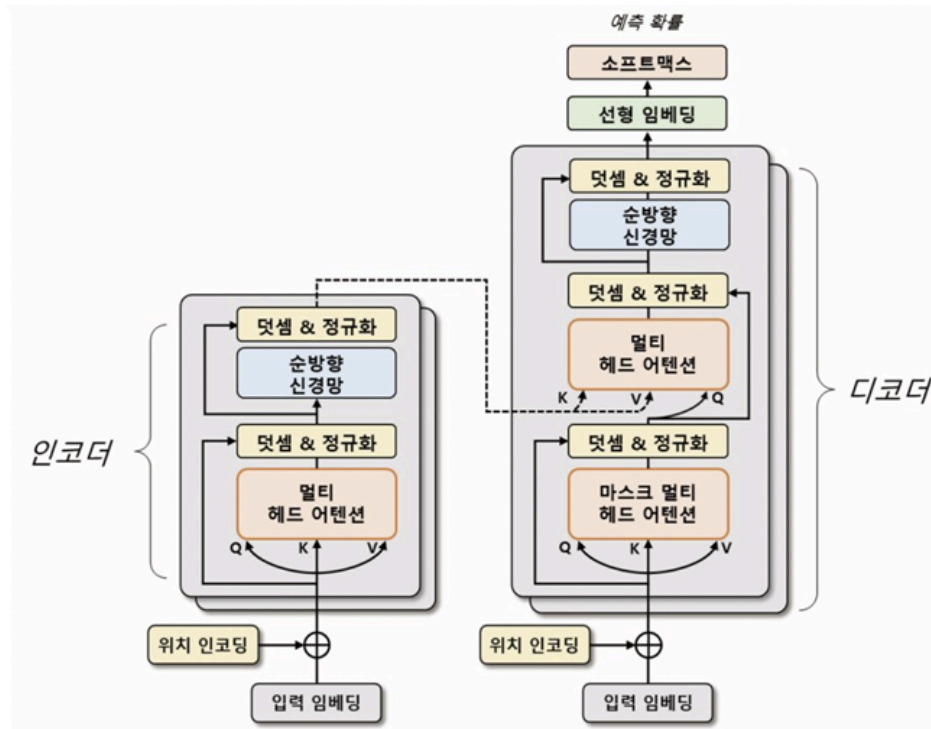


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

인코더+디코더 구조

- **Encoder:** 입력 sequence를 받아 문맥 정보를 포함한 representation을 생성
- **Decoder:** Encoder의 정보를 참고하면서 출력 sequence를 생성

입력 Embedding

- Transformer에서도 단어 자체를 그대로 모델에 넣는 것이 아니라 각 token을 Embedding Vector로 변환하여 입력함.
- 같은 단어는 기본적으로 같은 embedding을 갖기 때문에 embedding만으로는 sequence에서 해당 token이 몇 번째 위치에 등장했는지 알 수 없음.
- 따라서 Transformer에서는 embedding에 Positional Encoding을 더하여 위치 정보를 제공

Positional Encoding

- RNN은 입력을 순서대로 처리하기 때문에 순서 정보가 자연스럽게 포함됨.
- 반면 Transformer는 sequence 전체를 동시에 처리하기 때문에 별도의 장치가 없다면 단어의 순서를 알 수 없음.

- 이를 해결하기 위해 각 token embedding에 위치 정보를 나타내는 Positional Encoding을 추가

3. 특수 토큰 (Special Token) : 자연어 모델에서는 일반적인 단어 외에도 특정 기능을 수행하기 위한 token을 사용

BOS : Beginning of Sentence

- 문장의 시작을 표현

EOS : End of Sentence

- 문장의 끝을 표현
- Decoder가 언제 생성을 종료해야 하는지 판단할 수 있도록 함.

UNK : Unknown

- Vocabulary에 존재하지 않는 알 수 없는 단어를 나타냄.

PAD : Padding

- 서로 길이가 다른 문장을 동일한 길이의 batch로 처리하기 위해 빈 공간을 채우는 token

4. Attention

sequence 내에서 현재 처리하려는 token과 다른 token들이 얼마나 관련 있는지를 계산하고, 중요한 정보에 더 큰 가중치를 부여하는 방법

Query, Key, Value

각 입력 embedding은 서로 다른 Linear Transformation을 거쳐 세 종류의 vector로 변환됨.

Query

- 현재 token이 어떤 정보를 찾고 있는지를 나타냄.

Key

- 각각의 token이 어떤 정보를 가지고 있는지를 나타내는 일종의 기준 역할

Value

- Attention 결과에서 실제로 전달될 정보를 나타냄.

Attention Score

- Query와 각 Key를 비교하여 두 token이 얼마나 관련되어 있는지 계산
- 관련성이 높을수록 높은 Attention Score를 받음.
- 계산된 score에는 Softmax를 적용하여 각 token의 중요도를 나타내는 **Attention Weight**로 변환함.
- 마지막으로 각 Value에 해당 weight를 적용하여 가중합

Scaling

- Query와 Key의 차원이 커질수록 두 vector를 비교해서 얻은 값도 지나치게 커질 수 있음.
- 값이 너무 커지면 Softmax가 특정 값에 지나치게 집중하고 학습이 불안정해질 수 있음.
- 따라서 Attention Score의 크기를 적절하게 조절하는 **Scaling**을 적용함.

5. Self-Attention

- Self-Attention은 Query, Key, Value가 모두 같은 sequence에서 생성되는 Attention을 의미함.
- 각 단어가 같은 문장 안에 있는 다른 모든 단어를 참고하면서 자신의 representation을 새롭게 생성

즉, Self-Attention을 통해 각각의 token embedding은 단순히 하나의 단어 의미만 나타내는 것이 아니라 주변 문맥을 반영한 representation으로 변화함.

6. Multi-Head Attention

- 하나의 Attention만 사용하면 한 가지 관점에서 token의 관계를 학습하게 됨.
- Transformer에서는 Attention을 여러 개로 나누어 동시에 수행하는 **Multi-Head Attention**을 사용함.
- 각 Head는 서로 다른 Query, Key, Value projection을 사용함.

따라서 서로 다른 Head가 각각 문법적 관계, 단어 간 의미 관계, 장거리 dependency, 특정 위치 간 관계 등 다양한 특징에 집중할 수 있음.

각 Head의 Attention 결과를 마지막에 다시 결합하여 최종 representation을 생성함

Multi-Head Self-Attention

- 입력 sequence의 각 token이 sequence 내 다른 token들을 참고하여 문맥 정보를 학습
- Encoder의 Self-Attention에서는 Query, Key, Value가 모두 Encoder의 입력에서 생성

Feed-Forward Network

- Attention을 통해 얻은 각 token의 representation을 별도의 신경망을 통해 변환함.
- 동일한 Feed-Forward Network가 각 위치에 독립적으로 적용됨.
- Attention이 **token 사이의 관계**를 학습한다면 Feed-Forward Network는 각 token의 representation을 추가적으로 변환하는 역할을 함.

Residual Connection & Normalization

- 각 주요 sub-layer의 입력을 출력에 다시 더하는 Residual Connection을 사용함.
- 깊은 network에서 정보 및 gradient가 안정적으로 전달될 수 있도록 도움.
- 이후 Normalization을 통해 학습을 안정화

Transformer Decoder

Masked Multi-Head Self-Attention → Encoder-Decoder Attention → Feed-Forward Network

각 과정 사이에는 Residual Connection과 Normalization이 사용

Masked Self-Attention

- Decoder는 문장을 한 token씩 생성해야 함.
- 현재 token을 예측할 때 **아직 생성되지 않은 미래 token의 정보를 봐서는 안 됨.**
- 따라서 현재 위치보다 뒤쪽에 있는 token에 Attention하지 못하도록 **Mask**를 적용함.
- 미래 token에 해당하는 Attention Score를 매우 작은 값으로 만들어 Softmax 이후 사실상 선택되지 않도록 함.
- 이를 통해 Decoder가 이전까지 생성한 token만을 이용하여 다음 token을 예측하게 함.

Encoder-Decoder Attention

- Decoder에는 Self-Attention 외에도 Encoder가 생성한 입력 문장의 정보를 참고하는 Attention이 존재함.

- Decoder 쪽에서 Query를 만들고, Encoder의 출력에서 Key와 Value를 가져옴.
- 이를 통해 Decoder가 출력 단어를 생성할 때 입력 문장에서 어떤 부분을 참고해야 할지 결정할 수 있음.